XGBoost

Градиентный бустинг: регуляризация, вторые производные и скорость



Малышев Дмитрий / Боровиков Артём

Команда 30

НАВИГАЦИЯ

План доклада

01	Зачем нужен XGBoost	Имя Фамилия 1
02	Математика: цель, градиенты, сплиты	Имя Фамилия 1
03	Численный пример вручную	Имя Фамилия 2
04	Python и сравнение с альтернативами	Имя Фамилия 2

ЗАЧЕМ ЭТО НУЖНО

Какую задачу решает XGBoost

Табличные данные

- Скоринг, отток, прогноз спроса, CTR
- Разные масштабы и типы признаков
- Пропуски и нелинейности
- Нужны точность и скорость

17 из 29

победных решений Kaggle 2015 использовали XGBoost (Chen & Guestrin, KDD 2016)

2014 → 2016

от проекта Тяньци Чэня до статьи «XGBoost: A Scalable Tree Boosting System»

КОНТЕКСТ

Напоминание: градиентный бустинг

F_0

Начальный прогноз

→

g_i

Градиенты по ошибке

→

$h_m(x)$

Дерево на градиентах

→

F_m

$F_{m-1} + \eta \cdot h_m$

Слабые места классического GB

Только 1-я производная · нет регуляризации листьев · медленный поиск сплитов · пропуски нужно заполнять заранее

Регуляризованная цель

$$Obj = \sum I(y_i, \hat{y}_i) + \sum \Omega(f_k)$$

$$\Omega(f) = \gamma \cdot T + \frac{1}{2} \cdot \lambda \cdot \sum w_j^2$$

T — число листьев w_j — вес (значение) листа γ — цена за лист λ — L2-штраф весов

Разложение Тейлора до 2-го порядка

Для каждого объекта

$$g = \frac{\partial l}{\partial y}; \quad h = \frac{\partial^2 l}{\partial y^2}$$

Для MSE:

$$l = \frac{1}{2}(y_i - \hat{y}_i)^2, \quad g_i = \hat{y}_i - y_i, \quad h_i = 1$$

Вес листа j

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

Качество структуры

$$Obj^* = -\frac{1}{2}\sum_{j=1}^{T}\frac{G_j^2}{H_j + \lambda} + \gamma T$$

МАТЕМАТИКА

Выбор сплита: Gain

$$Gain = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

Gain > 0

сплит улучшает цель → делаем

Gain < 0

сплит не окупает γ → ветка обрезается

λ ↑ или γ ↑

меньше сплитов, проще дерево

СКОРОСТЬ

Что делает XGBoost быстрым

1

Гистограммы

tree_method=hist: значения в корзины, не перебираем все пороги

2

Пропуски

Sparsity-aware: узел сам выбирает направление для NaN

3

Параллелизм

Поиск лучшего сплита по признакам идёт параллельно

4

Память и GPU

Блоки с учётом кэша, out-of-core, device=cuda

Данные, градиенты, кандидаты сплитов

x, y, $F_0 = 4$, градиент $g = F_0 - y$, гессиан $h = 1$

x	y	g	h
1	1	+3	1
2	2	+2	1
3	6	−2	1
4	7	−3	1

$$Gain = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right] - \gamma, \quad \lambda = 1$$

порог	G_L / H_L	G_R / H_R	Gain
x < 1,5	+3 / 1	−3 / 3	3,375
x < 2,5	**+5 / 2**	**−5 / 2**	**8,333**
x < 3,5	+3 / 3	−3 / 1	3,375

Лучший сплит: x < 2,5 (Gain = 8,33)

Веса листьев и обновление прогноза

Веса листьев

$$w_L = -\frac{5}{2+1} = -1.667$$

$$w_R = \frac{5}{2+1} = 1.667$$

$$F_m = F_0 + \eta \cdot w$$

$$\eta = 0.5$$

x	y	F_0	F_1
1	1	4	3,167
2	2	4	3,167
3	6	4	4,833
4	7	4	4,833

6,50

MSE до

3,03

MSE после

PYTHON · src/03_xgboost_example.py

Пример: классификация на breast_cancer

```
import xgboost as xgb

model = xgb.XGBClassifier(
    n_estimators=1000, learning_rate=0.05,
    max_depth=3, reg_lambda=1.0,
    subsample=0.8, colsample_bytree=0.8,
    early_stopping_rounds=30,
    eval_metric="logloss")

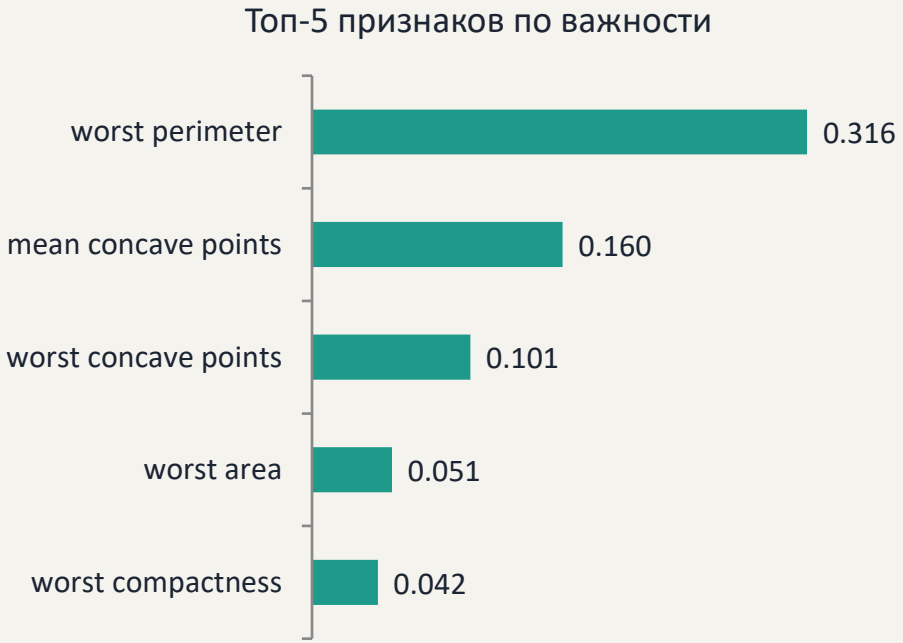
model.fit(Xtr, ytr, eval_set=[(Xval, yval)],
          verbose=False)
print(model.best_iteration)
```

Результат на тестовой выборке

157 лучшая итерация (из 1000)

0,947 accuracy

0,981 ROC-AUC



Ключевые гиперпараметры

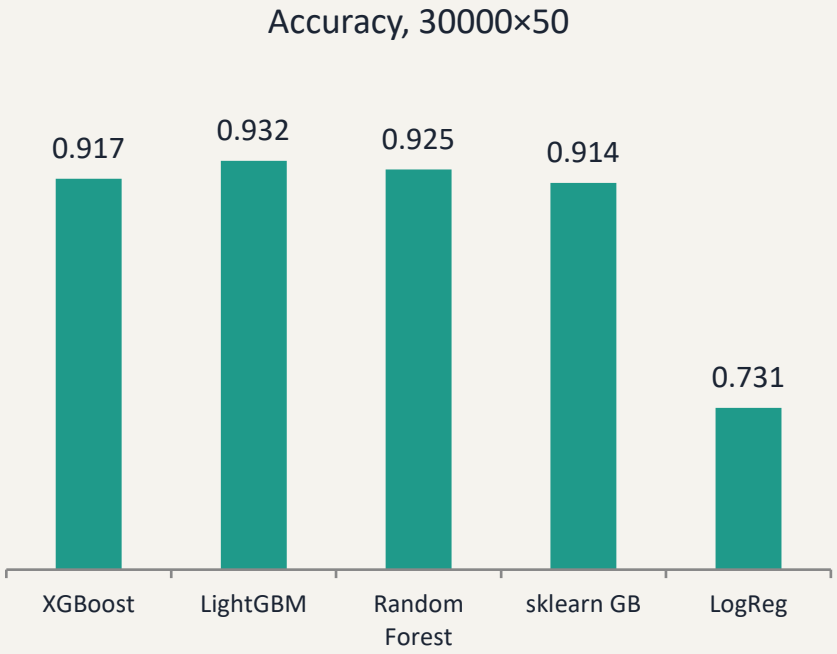
Параметр	Что делает	Типично
learning_rate (η)	Шаг обучения; меньше — точнее, но больше деревьев	0,01–0,3
max_depth	Глубина дерева — сложность модели	3–8
reg_lambda / reg_alpha	L2 / L1-штраф весов листьев	1 / 0
gamma	Минимальный Gain для сплита	0+
subsample / colsample_bytree	Доля объектов / признаков на дерево	0,6–1,0
n_estimators + early_stopping	Число деревьев ограничивает ранняя остановка	100–1000+

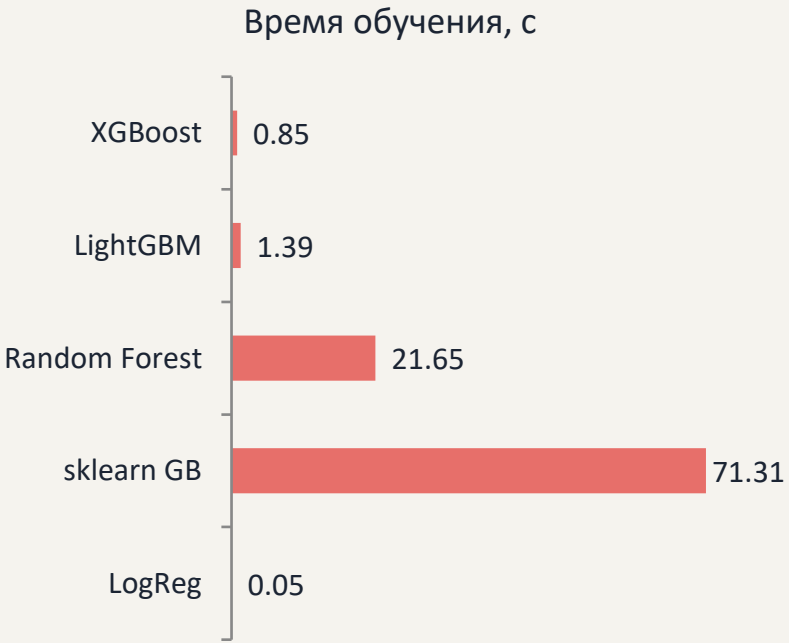
АЛЬТЕРНАТИВЫ

XGBoost и конкуренты

Метод	Особенности	Слабая сторона
XGBoost	2-й порядок, L1/L2 и γ, hist, GPU	Нужен тюнинг; категории — экспериментально
LightGBM	Рост по листьям, GOSS, EFB	Переобучается на малых данных
CatBoost	Нативные категории, ordered boosting	Часто медленнее обучается
sklearn GB	Простой, 1-й порядок, без регуляризации листьев	Медленный на больших данных
Random Forest	Независимые деревья, мало тюнинга	Точность ниже бустинга, тяжёлые модели

Эксперимент: точность и время





Когда брать XGBoost

Подходит

- Табличные данные
- Пропуски, нелинейности
- Нужны качество и скорость

Осторожно

- Изображения, текст, звук
- Нужен тюнинг и ранняя остановка
- Не экстраполирует тренды

Спасибо! Вопросы?